

23. ROS2 TF2 Coordinate Transformation

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. TF2 Overview

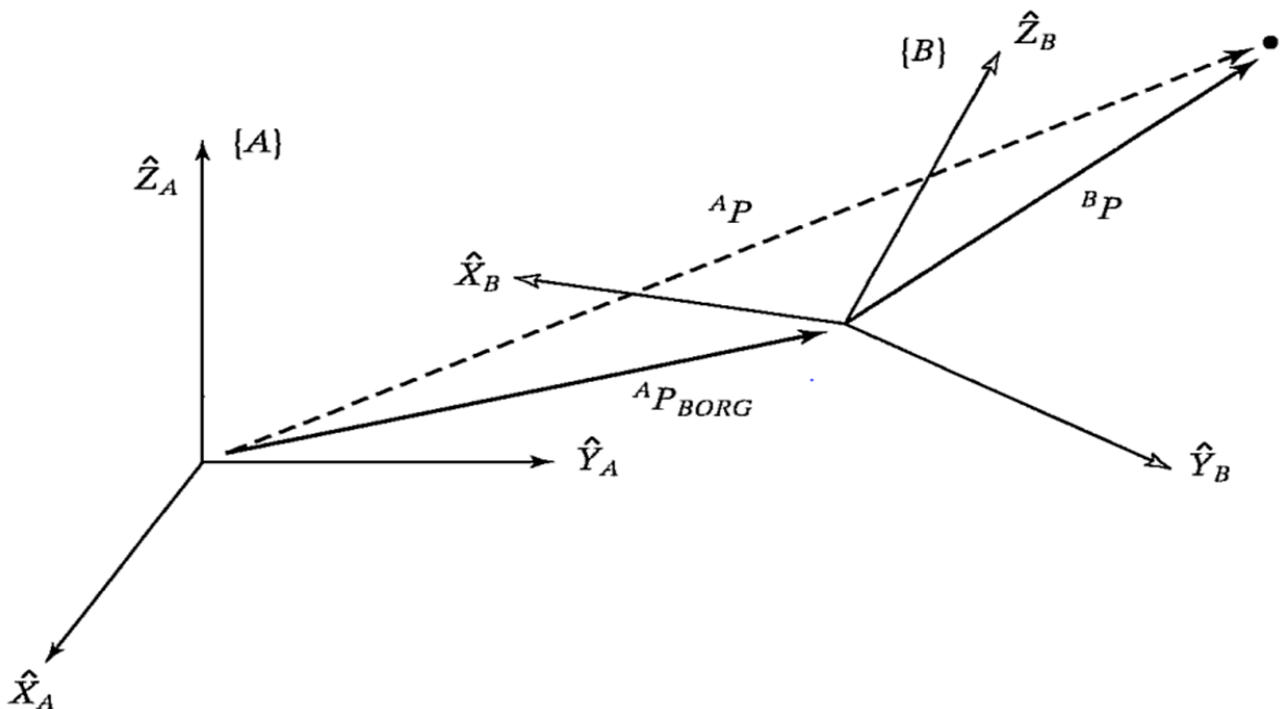
The coordinate system is a very familiar concept and an important foundation in robotics. In a complete robot system, there are many coordinate frames. How should we manage the positional relationships between these coordinate frames? ROS provides us with a powerful tool for managing coordinate frames: TF2.

TF system reference: [tf: The transform library](#) | [IEEE Conference Publication](#) | [IEEE Xplore](#)

2. Coordinate Frames in Robots

In mobile robot systems, coordinate frames are equally crucial. For example, a mobile robot's center point is the base coordinate frame (Base Link). The position where the LiDAR is located is called the LiDAR coordinate frame (laser link). When the robot moves, the odometer accumulates position. This position's reference frame is called the odometer coordinate frame (odom). The odometer also has accumulated errors and drift. The reference frame for absolute position is called the map coordinate frame (map).

The relationships between coordinate frames at each level are complex. Some are relatively fixed, while others are constantly changing. Simple coordinate frames become complex within a spatial range, making a good coordinate frame management system particularly important.



The basic theory of coordinate frame transformation relationships is explained in every robotics textbook. It can be decomposed into two parts: **translation and rotation**, described by a 4x4 matrix. Draw coordinate frames in space, and the transformation relationship between them is actually a vector's mathematical description.

The underlying principle of TF functionality in ROS is the encapsulation of these mathematical transformations. For detailed theoretical knowledge, you can refer to robotics textbooks. We will mainly explain how to use the TF coordinate management system.

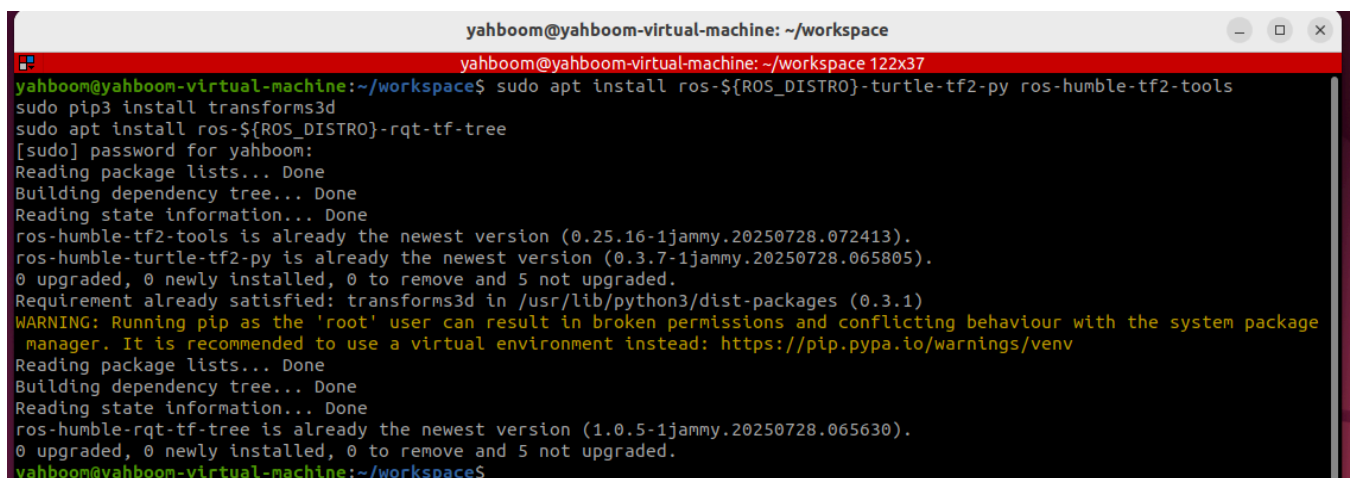
3. TF Command Line Operations

First, through the example of two turtles, let's understand a robot following algorithm based on coordinate frames. **For easier demonstration, it's best to operate this section's course in a virtual machine.**

3.1. Install Related Tools

This example requires installing the corresponding packages, tf turtle simulator example, and tf tree visualization tool:

```
sudo apt install ros-${ROS_DISTRO}-turtle-tf2-py ros-jazzy-tf2-tools
sudo pip3 install transforms3d --break-system-packages
sudo apt install ros-${ROS_DISTRO}-rqt-tf-tree
```

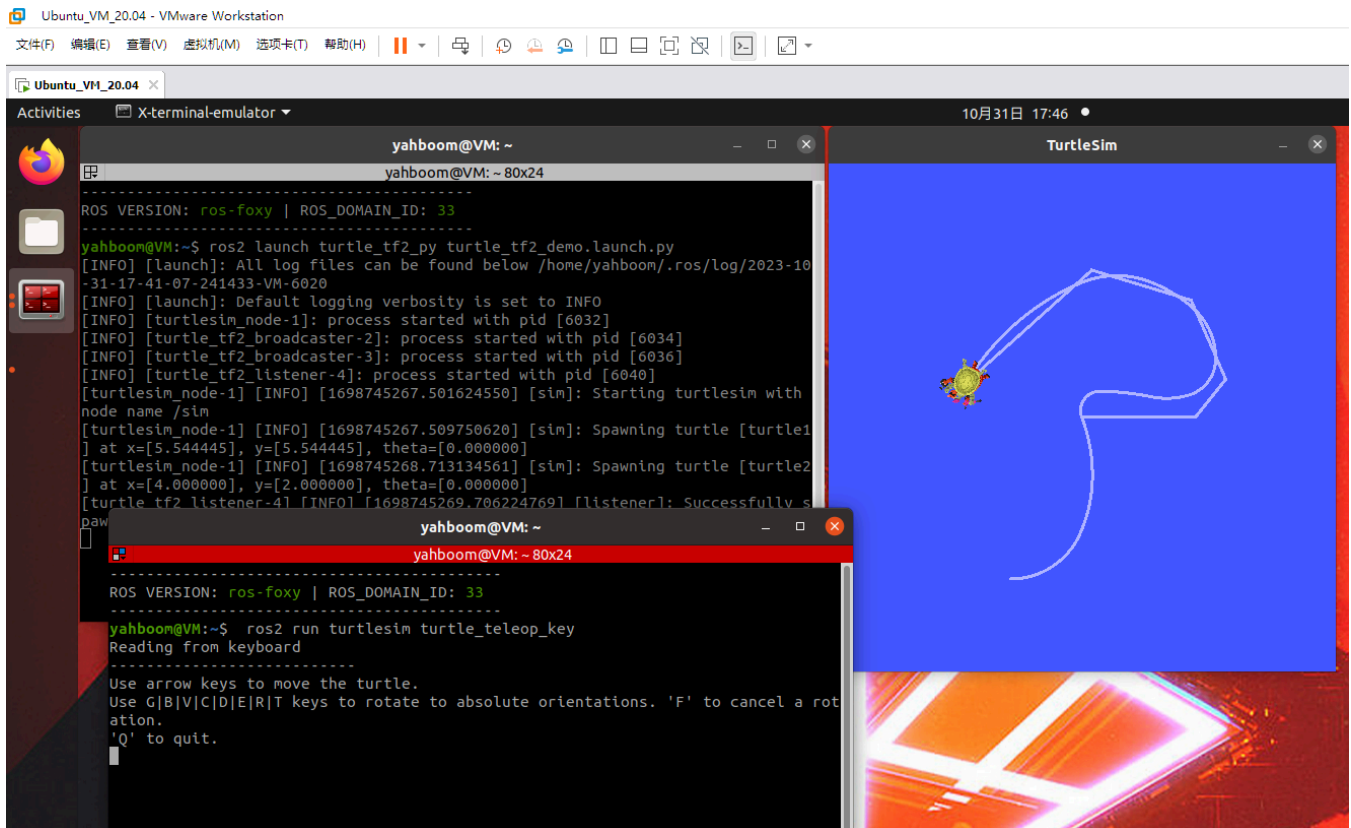
A terminal window titled 'yahboom@yahboom-virtual-machine: ~/workspace' showing the installation of ROS packages. The user runs 'sudo apt install ros-\${ROS_DISTRO}-turtle-tf2-py ros-humble-tf2-tools', followed by 'sudo pip3 install transforms3d' and 'sudo apt install ros-\${ROS_DISTRO}-rqt-tf-tree'. The terminal output shows the progress of the installation, including package lists, dependency trees, and state information. A warning message is displayed: 'WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv'. The installation of 'transforms3d' is successful, and the installation of 'rqt-tf-tree' is also successful. The terminal ends with the prompt 'yahboom@yahboom-virtual-machine:~/workspace\$'.

3.2. Start

Then you can start through a launch file. After that, we can control one turtle, and the other turtle will automatically follow.

```
ros2 launch turtle_tf2_py turtle_tf2_demo.launch.py
ros2 run turtlesim turtle_teleop_key
```

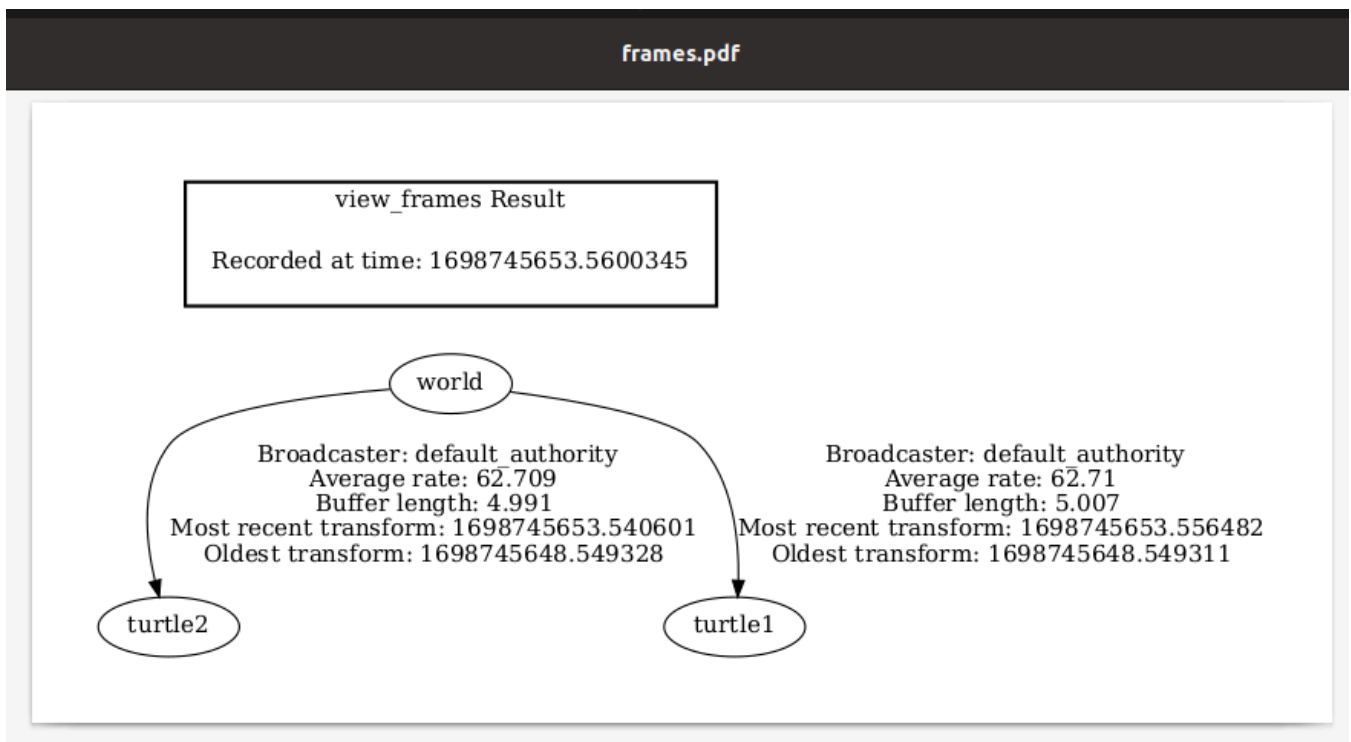
When we control one turtle to move, the other turtle will also follow.



3.3. View TF Tree

```
ros2 run rqt_tf_tree rqt_tf_tree
```

You can see the TF transform tree in the rqt window.

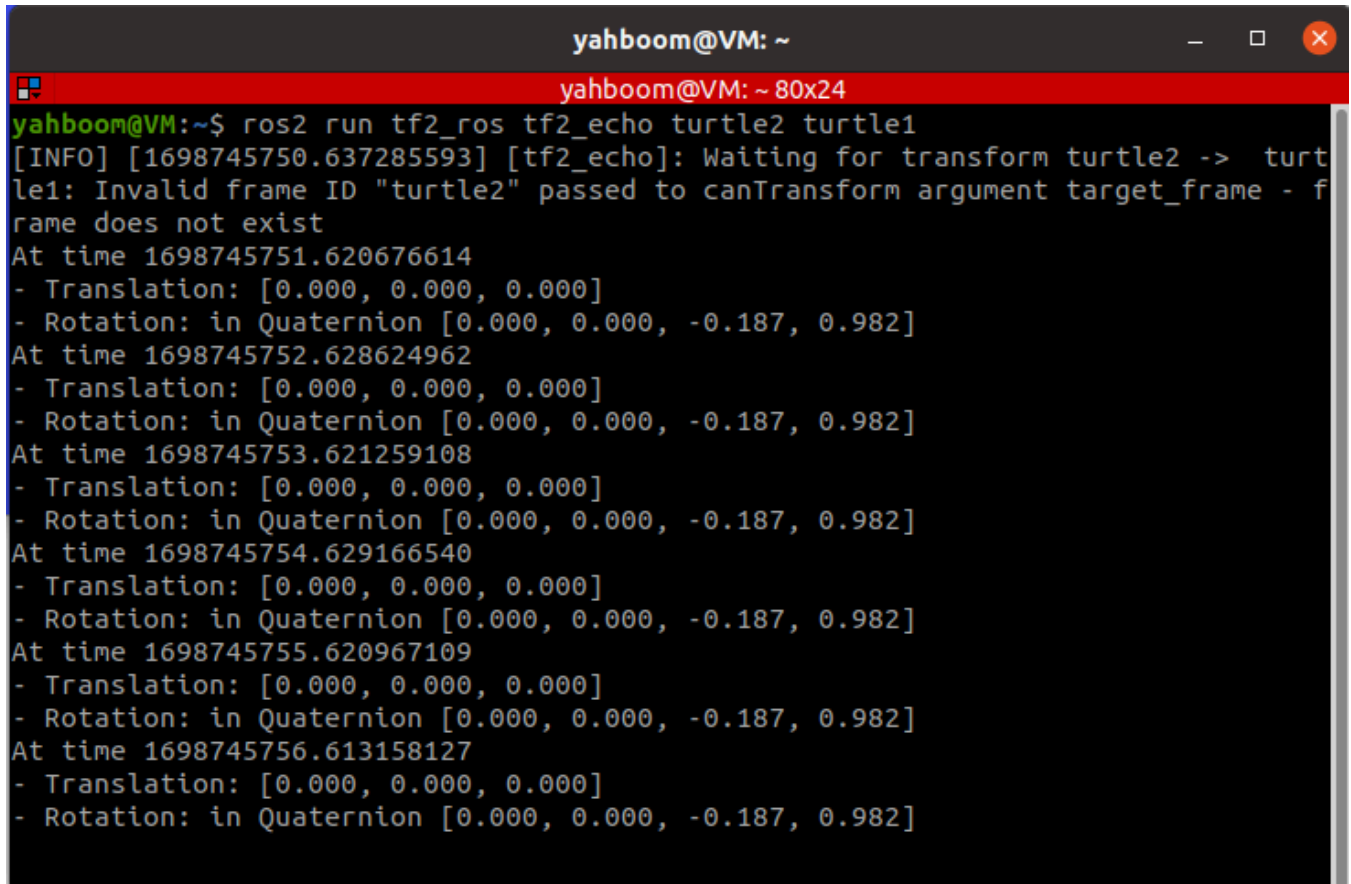


3.4. Query Coordinate Transform Information

Just seeing the structure of coordinate frames is not enough. If we want to know the specific relationship between two coordinate frames, we can use the `tf2_echo` tool:

```
ros2 run tf2_ros tf2_echo turtle2 turtle1
```

After successful execution, the terminal will continuously print the coordinate frame transformation values.

A terminal window titled 'yahboom@VM: ~' with a red title bar. The terminal shows the command 'ros2 run tf2_ros tf2_echo turtle2 turtle1' being executed. The output starts with an INFO message: '[INFO] [1698745750.637285593] [tf2_echo]: Waiting for transform turtle2 -> turtle1: Invalid frame ID "turtle2" passed to canTransform argument target_frame - frame does not exist'. This is followed by a series of transformation data points at regular intervals. Each point starts with 'At time' followed by a timestamp, then '- Translation: [0.000, 0.000, 0.000]' and '- Rotation: in Quaternion [0.000, 0.000, -0.187, 0.982]'.

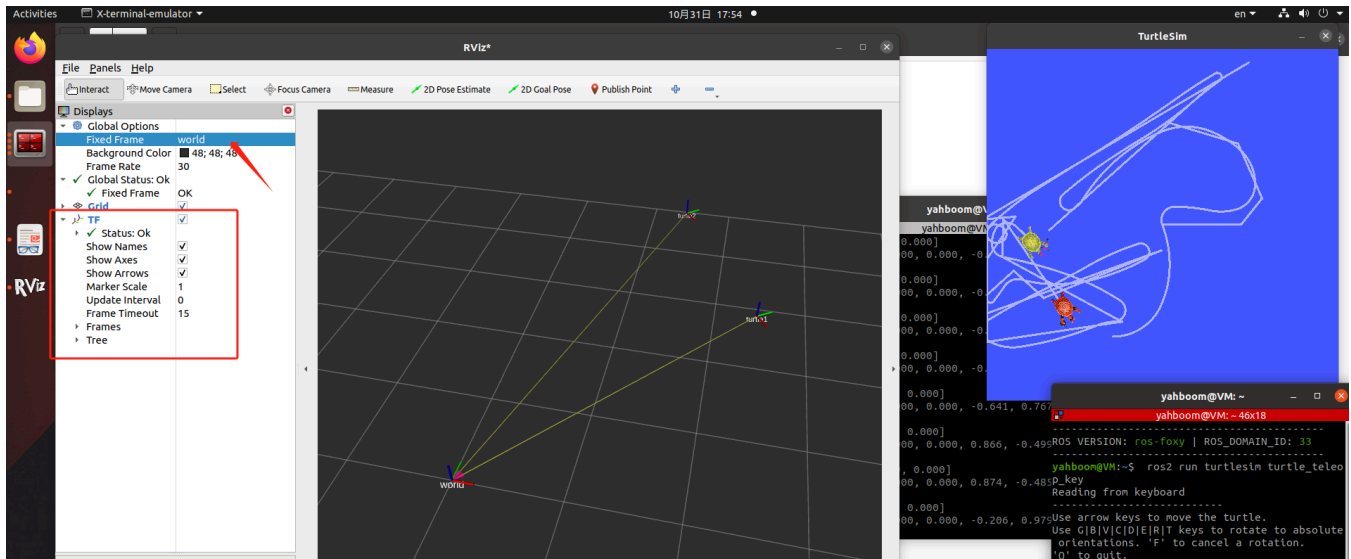
```
yahboom@VM: ~  
yahboom@VM: ~ 80x24  
yahboom@VM:~$ ros2 run tf2_ros tf2_echo turtle2 turtle1  
[INFO] [1698745750.637285593] [tf2_echo]: Waiting for transform turtle2 -> turtle1: Invalid frame ID "turtle2" passed to canTransform argument target_frame - frame does not exist  
At time 1698745751.620676614  
- Translation: [0.000, 0.000, 0.000]  
- Rotation: in Quaternion [0.000, 0.000, -0.187, 0.982]  
At time 1698745752.628624962  
- Translation: [0.000, 0.000, 0.000]  
- Rotation: in Quaternion [0.000, 0.000, -0.187, 0.982]  
At time 1698745753.621259108  
- Translation: [0.000, 0.000, 0.000]  
- Rotation: in Quaternion [0.000, 0.000, -0.187, 0.982]  
At time 1698745754.629166540  
- Translation: [0.000, 0.000, 0.000]  
- Rotation: in Quaternion [0.000, 0.000, -0.187, 0.982]  
At time 1698745755.620967109  
- Translation: [0.000, 0.000, 0.000]  
- Rotation: in Quaternion [0.000, 0.000, -0.187, 0.982]  
At time 1698745756.613158127  
- Translation: [0.000, 0.000, 0.000]  
- Rotation: in Quaternion [0.000, 0.000, -0.187, 0.982]
```

3.5. Coordinate Frame Visualization

- Run `rviz2`, then add TF display plugin:

```
rviz2
```

In `rviz2`, set the reference coordinate frame to: `world`, add TF display, and then let the turtle move. The coordinate axes in `Rviz` will start moving. Isn't this more intuitive?



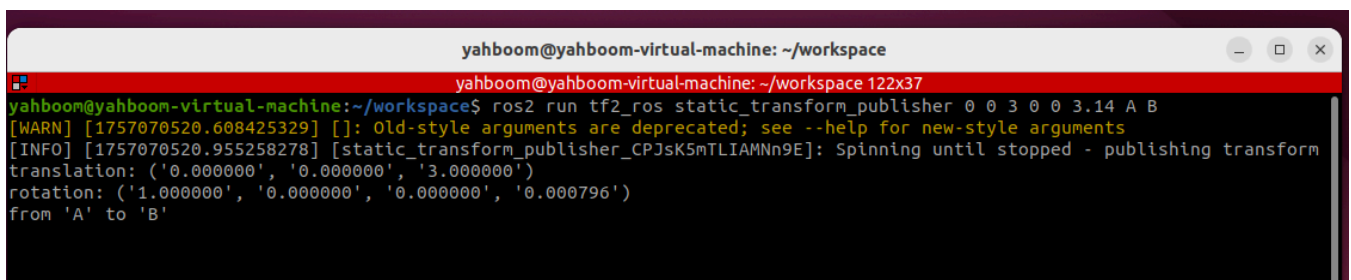
4. Static Coordinate Transformation

Static coordinate transformation refers to the relative position between two coordinate frames being fixed. For example, the position between the LiDAR and base_link is fixed.

Example: **For easier demonstration, it's best to operate this section's course in a virtual machine.**

4.1. Publish Pose from A to B

```
ros2 run tf2_ros static_transform_publisher 0 0 3 0 0 3.14 A B
```



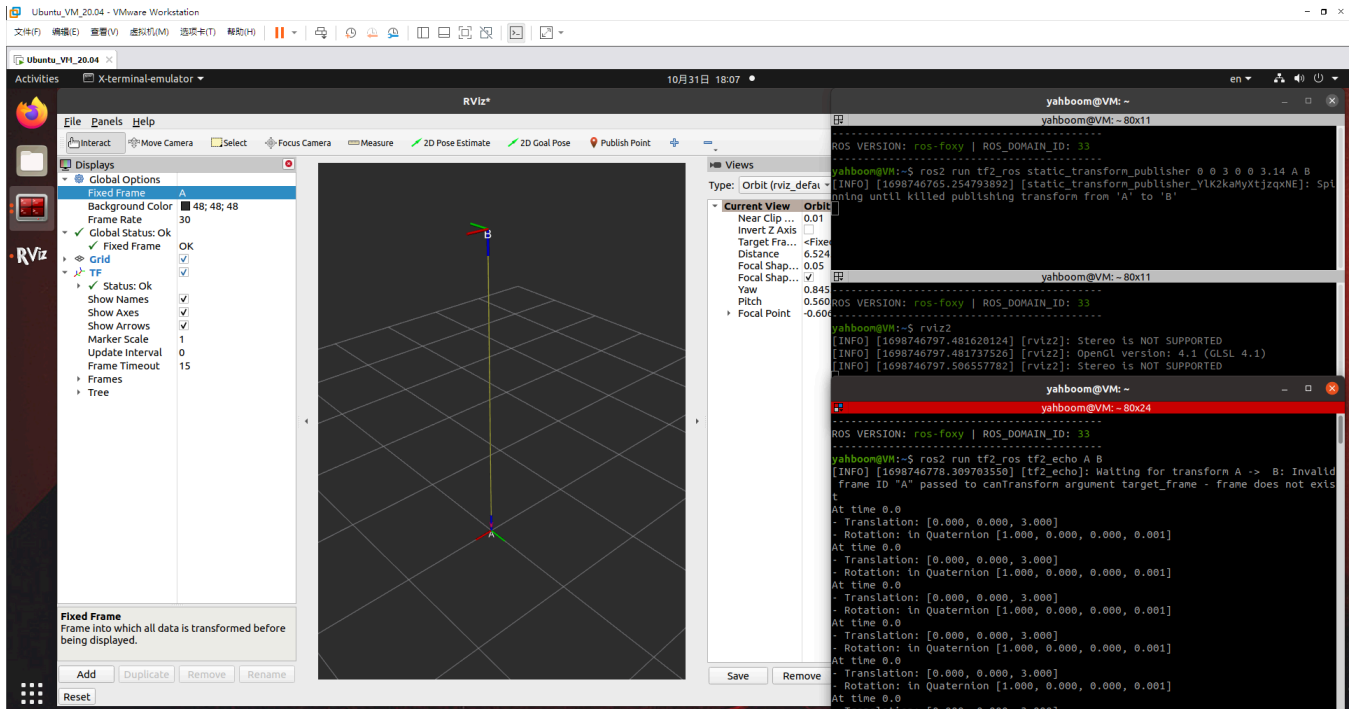
4.2. Listen/Get TF Relationship

```
ros2 run tf2_ros tf2_echo A B
```

4.3. Rviz Visualization

- Run rviz2, then add TF display plugin:

```
rviz2
```



5. Example Introduction

The previous section explained the TF relationships in the turtle following example provided by the system. This section we implement this functionality ourselves.

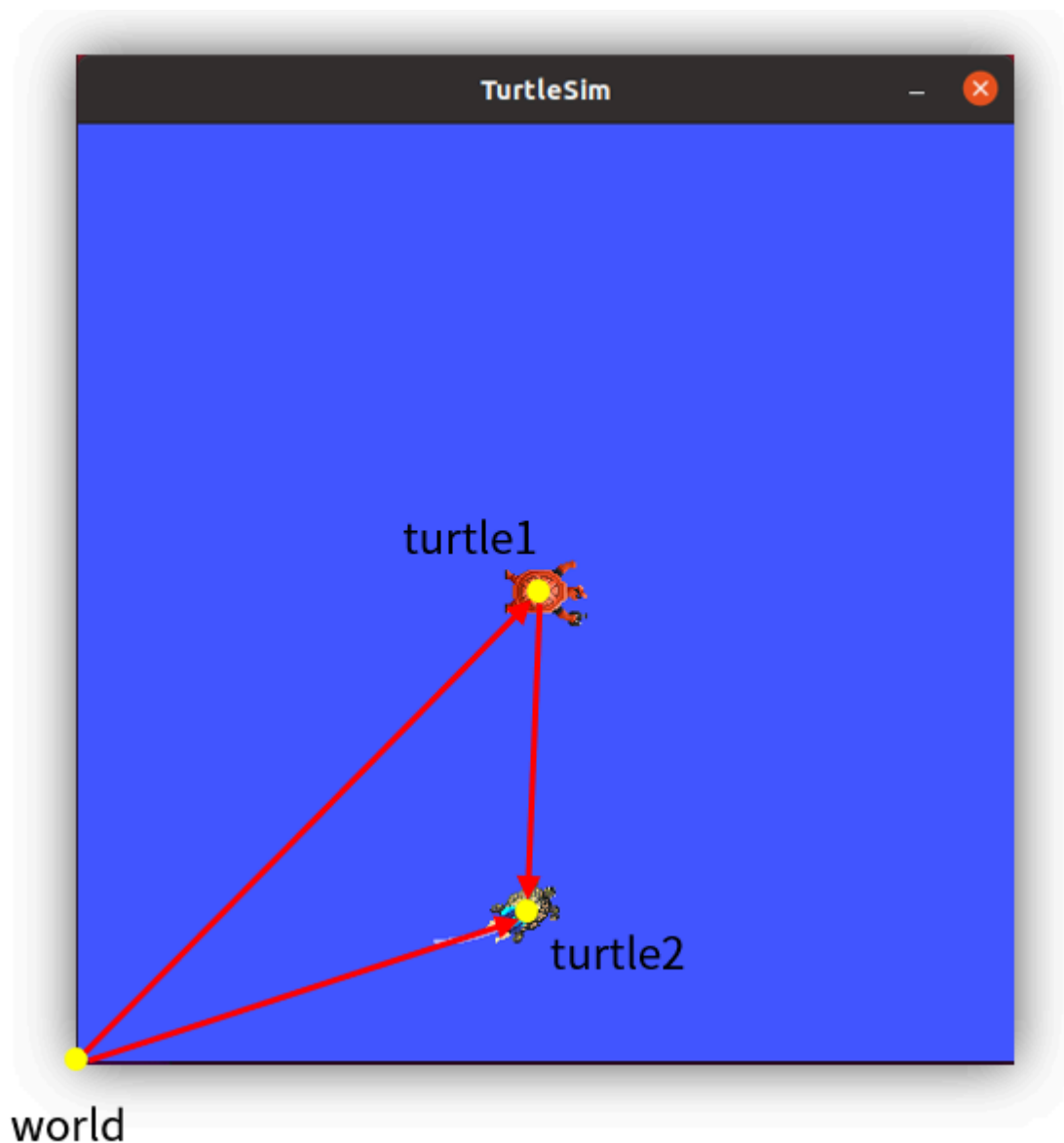
Course content:

- Implement the turtle following example through programming
- Master implementing dynamic broadcasters through programming
- Master listening to coordinate transformations between coordinate frames through programming
- Master PID control to convert physical quantities (distance, angle) into velocity control values

Advanced content:

- Understand TF system's cross-time-dimension coordinate frame transformation functionality

6. Turtle Following Example Implementation Principle Analysis



In the two-turtle simulator, we can define three coordinate frames. For example, the simulator's global reference frame is called world. The turtle1 and turtle2 coordinate frames are at the center points of the two turtles. This way, the relative position of turtle1 and the world coordinate frame can represent turtle 1's position. The same applies to turtle 2.

To make turtle 2 move toward turtle 1, we draw a line between them and add an arrow. Does this remind you of vector calculations from high school? We said the description method for coordinate transformations is vectors. So in this following example, TF can solve this well.

The vector's length represents distance, and the direction represents angle. With distance and angle, we can set a time arbitrarily and calculate velocity. Then it's just the encapsulation and publishing of velocity topics, and turtle 2 can start moving.

Therefore, the core of this example is implementing vector calculations through coordinate frames. The two turtles keep moving, so this vector also needs to be calculated periodically. This requires TF's dynamic broadcasting and listening.

7. Create Package

1. Create a new package in the workspace src directory to store our files:

```
ros2 pkg create pkg_tf --build-type ament_python --dependencies rclpy --node-name
turtle_tf_broadcaster
```

After executing the above command, the pkg_tf package will be created, along with a turtle_tf_broadcaster node, and the related configuration files will already be set up. Add the following code to the [turtle_tf_broadcaster.py] file:

```
import rclpy                                # ROS2 Python interface library
from rclpy.node import Node                 # ROS2 Node class
from geometry_msgs.msg import TransformStamped # Transform message
import tf_transformations                    # TF coordinate transformation library
from tf2_ros import TransformBroadcaster    # TF coordinate transformation
broadcaster                                # turtlesim turtle position message

from turtlesim.msg import Pose

class TurtleTFBroadcaster(Node):

    def __init__(self, name):
        super().__init__(name)              # ROS2 node parent class
        initialization

        self.declare_parameter('turtlename', 'turtle') # Create a turtle name
        parameter
        self.turtlename = self.get_parameter(
            value, otherwise use default
            'turtlename').get_parameter_value().string_value

        self.tf_broadcaster = TransformBroadcaster(self) # Create and initialize TF
        broadcast object

        self.subscription = self.create_subscription(
            position messages
            Pose,
            f'/{self.turtlename}/pose',      # Use turtle name obtained
            from parameter
            self.turtle_pose_callback, 1)

    def turtle_pose_callback(self, msg):      # Callback to
        transform position messages to coordinate transforms
        transform = TransformStamped()        # Create transform
        message object

        transform.header.stamp = self.get_clock().now().to_msg() # Set timestamp for
        transform message
        transform.header.frame_id = 'world'    # Set source
        coordinate frame for transform
```

```

        transform.child_frame_id = self.turtlename # Set target
coordinate frame for transform
        transform.transform.translation.x = msg.x # Set X, Y, Z
translation in transform
        transform.transform.translation.y = msg.y
        transform.transform.translation.z = 0.0
        q = tf_transformations.quaternion_from_euler(0, 0, msg.theta) # Convert Euler angles
to quaternion (roll, pitch, yaw)
        transform.transform.rotation.x = q[0] # Set X, Y, Z rotation
in transform (quaternion)
        transform.transform.rotation.y = q[1]
        transform.transform.rotation.z = q[2]
        transform.transform.rotation.w = q[3]

        # Send the transformation
        self.tf_broadcaster.sendTransform(transform) # Broadcast transform, update
transform info when turtle position changes

def main(args=None):
    rclpy.init(args=args) # ROS2 Python interface
initialization
    node = TurtleTFBroadcaster("turtle_tf_broadcaster") # Create and initialize ROS2 node
object
    rclpy.spin(node) # Loop waiting for ROS2 exit
    node.destroy_node() # Destroy node object
    rclpy.shutdown() # Shutdown ROS2 Python interface

```

2. Next, create a [turtle_following.py] file in the same directory as turtle_tf_broadcaster.py and add the following code:

```

import math
import rclpy # ROS2 Python interface library
from rclpy.node import Node # ROS2 Node class
import tf_transformations # TF coordinate transformation
library
from tf2_ros import TransformException # TF transform exception class
from tf2_ros.buffer import Buffer # Buffer class for storing
transform information
from tf2_ros.transform_listener import TransformListener # Transform listener class
from geometry_msgs.msg import Twist # ROS2 velocity control message
from turtlesim.srv import Spawn # Turtle spawn service interface
class TurtleFollowing(Node):

    def __init__(self, name):
        super().__init__(name) # ROS2 node parent class
initialization

        self.declare_parameter('source_frame', 'turtle1') # Create source frame
name parameter
        self.source_frame = self.get_parameter(
parameter value, otherwise use default

```



```

        trans.transform.translation.y ** 2)

        self.publisher.publish(msg)                # Publish velocity
command, turtle follows motion
    else:                                           # If following turtle has
not been spawned                                # Check if turtle has
                                                # been spawned
        if self.result.done():
            self.get_logger().info(
                f'Successfully spawned {self.result.result().name}')
            self.turtle_spawned = True
        else:                                     # Following turtle still
not spawned
            self.get_logger().info('Spawn is not finished')
        else:                                     # If turtle spawning
service has not been requested
            if self.spawner.service_is_ready():    # If turtle spawning
server is ready
                request = Spawn.Request()          # Create request data
                request.name = 'turtle2'           # Set request data
content, including turtle name, xy position, pose
                request.x = float(4)
                request.y = float(2)
                request.theta = float(0)

                self.result = self.spawner.call_async(request) # Send service request
                self.turtle_spawning_service_ready = True      # Set flag indicating
request has been sent
            else:
                self.get_logger().info('Service is not ready') # Prompt that turtle
spawning server is not ready

def main(args=None):
    rclpy.init(args=args)                        # ROS2 Python interface initialization
    node = TurtleFollowing("turtle_following")    # Create and initialize ROS2 node object
    rclpy.spin(node)                             # Loop waiting for ROS2 exit
    node.destroy_node()                          # Destroy node object
    rclpy.shutdown()                             # Shutdown ROS2 Python interface

```

3. Create a launch folder under the pkg_tf package. Create a [turtle_following.launch.py] file in the launch folder and add the following content:

```

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():

    return LaunchDescription([

```

```

        DeclareLaunchArgument('source_frame', default_value='turtle1', description='Target
frame name.'),
        Node(
            package='turtlesim',
            executable='turtlesim_node',
        ),
        Node(
            package='pkg_tf',
            executable='turtle_tf_broadcaster',
            name='broadcaster1',
            parameters=[
                {'turtlename': 'turtle1'}
            ]
        ),
        Node(
            package='pkg_tf',
            executable='turtle_tf_broadcaster',
            name='broadcaster2',
            parameters=[
                {'turtlename': 'turtle2'}
            ]
        ),
        Node(
            package='pkg_tf',
            executable='turtle_following',
            name='listener',
            parameters=[
                {'source_frame': LaunchConfiguration('source_frame')}
            ]
        ),
    ],
)

```

8. Edit Configuration File

8.1. Configure in setup.py

- Import related libraries:

```

import os
from glob import glob

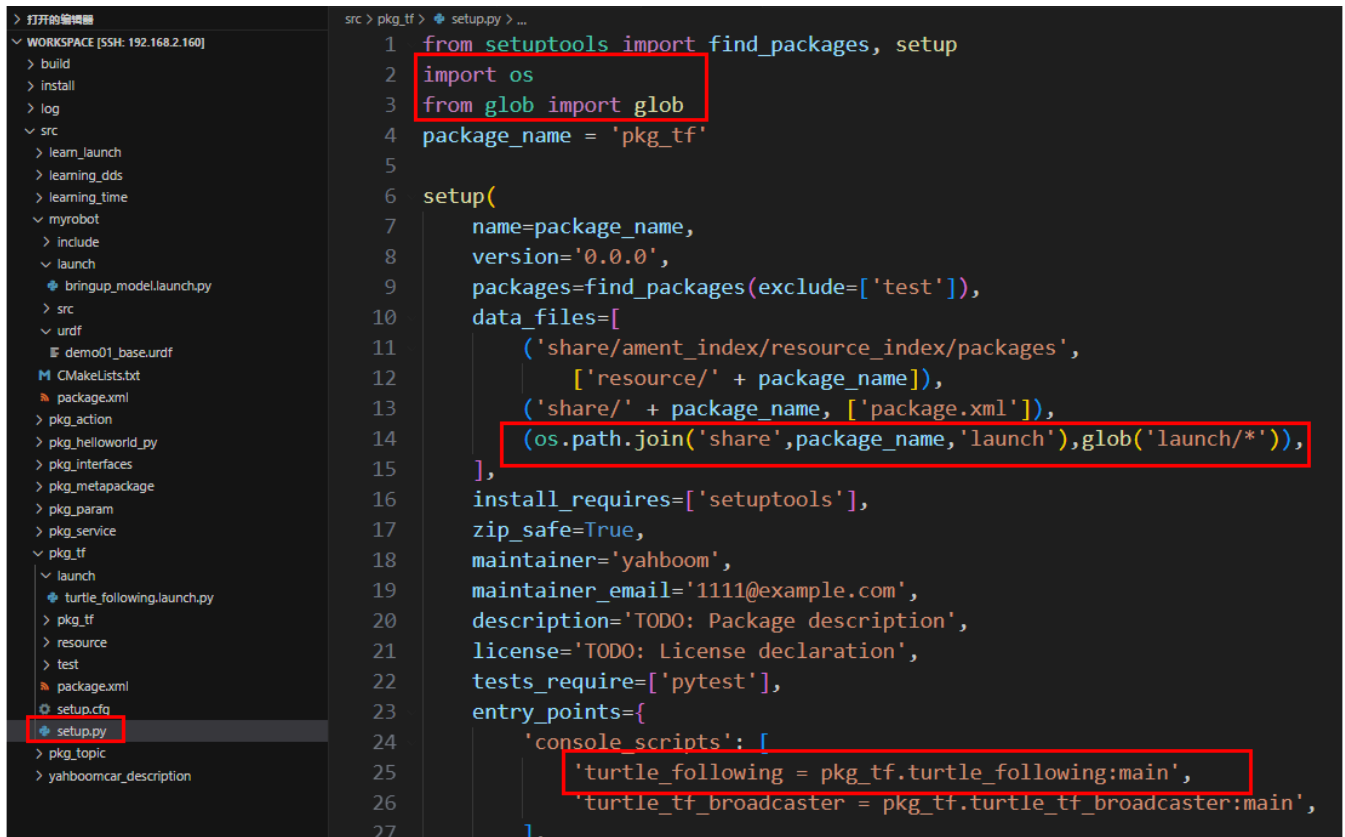
```

- Add turtle_following node information and add command to copy launch files to install's shared directory:

```

(os.path.join('share', package_name, 'launch'), glob('launch/*')),

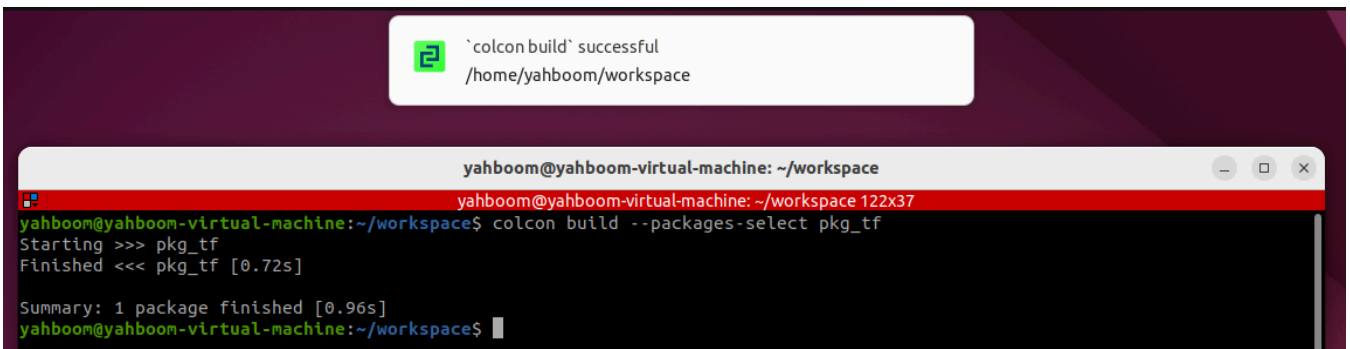
```



```
src > pkg_tf > setup.py > ...
1  from setuptools import find_packages, setup
2  import os
3  from glob import glob
4  package_name = 'pkg_tf'
5
6  setup(
7      name=package_name,
8      version='0.0.0',
9      packages=find_packages(exclude=['test']),
10     data_files=[
11         ('share/ament_index/resource_index/packages',
12          ['resource/' + package_name]),
13         ('share/' + package_name, ['package.xml']),
14         (os.path.join('share', package_name, 'launch'), glob('launch/*')),
15     ],
16     install_requires=['setuptools'],
17     zip_safe=True,
18     maintainer='yahboom',
19     maintainer_email='1111@example.com',
20     description='TODO: Package description',
21     license='TODO: License declaration',
22     tests_require=['pytest'],
23     entry_points={
24         'console_scripts': [
25             'turtle_following = pkg_tf.turtle_following:main',
26             'turtle_tf_broadcaster = pkg_tf.turtle_tf_broadcaster:main',
27         ],
28     },
29 )
```

9. Build Package

```
colcon build --packages-select pkg_tf
```



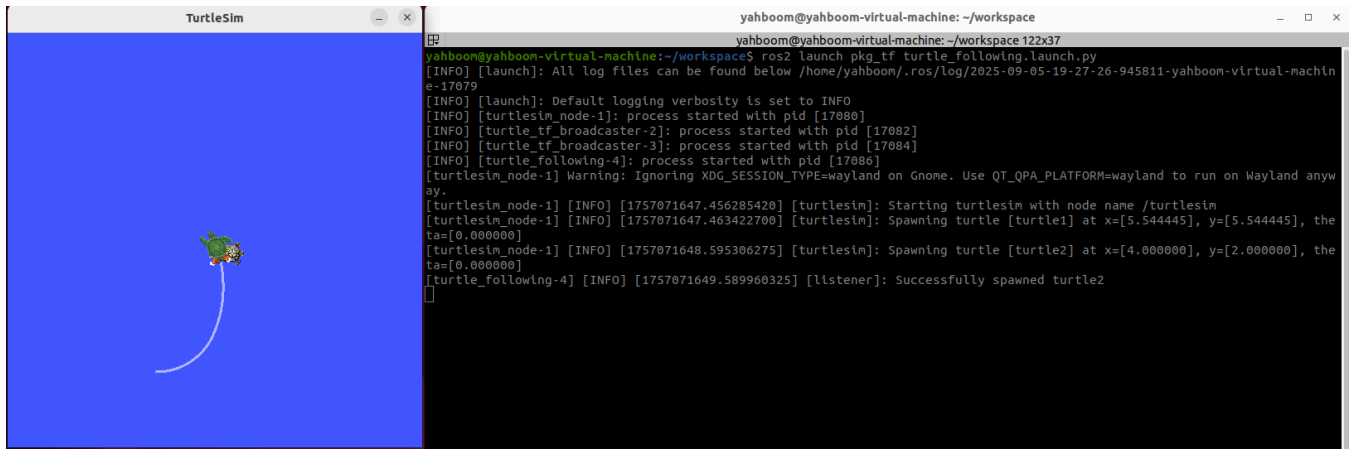
```
yahboom@yahboom-virtual-machine: ~/workspace
colcon build --packages-select pkg_tf
Starting >>> pkg_tf
Finished <<< pkg_tf [0.72s]

Summary: 1 package finished [0.96s]
yahboom@yahboom-virtual-machine: ~/workspace$
```

10. Run Program

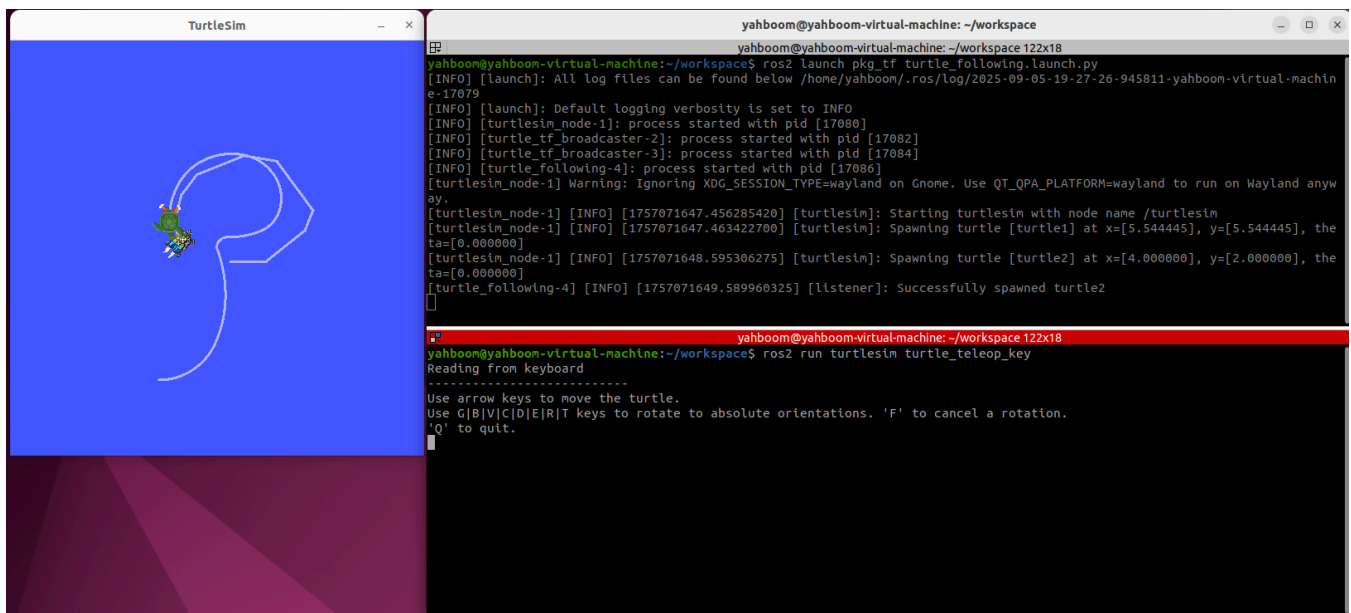
- Refresh terminal environment variables, then run:

```
ros2 launch pkg_tf turtle_following.launch.py
```



- Start turtle keyboard control node. Control the first turtle's motion, and the second turtle will automatically follow:

```
ros2 run turtlesim turtle_teleop_key
```



In this terminal, pressing the arrow keys can control one turtle to move, and then the other turtle will follow until they coincide.

11. Advanced Content

- Understand TF's cross-time-dimension transformation capability

Buffer can automatically cache all TF transformation relationships within the TF system for the past 10 seconds (the cache duration can be set arbitrarily through Buffer construction parameters). All transformations in the buffer are timestamped, and all transformations are continuously traceable. Even if two coordinate frames are at different time points, the transformation relationship between them can still be found. For this aspect, you can refer to the TF system design reference. There are detailed principles explained (the reference is in this section's course folder, or through the link at the beginning of this section).

```
self.tf_buffer = Buffer() # 创建保存坐标变换信息的缓冲区
self.tf_listener = TransformListener(self.tf_buffer, self) # 创建坐标变换的监听器
```

- The following is a supplement to our turtle following example. The red arrows indicate the ability to query transformations between two coordinate frames at different time points across time.



Fig. 4: A simple *tf* tree from a core ROS tutorial, with debugging information.

Long Term Data Storage: The above example assumes that you have the ability to transform between frames a and b at time 0 and b and c at time 1. This will only work if the length of the buffer in your *Listener* module is long enough to encompass both time 0 and time 1. To be able to store data for a long time, it should be transformed into the fixed frame, b, when saved. Then the source frame is the same as frame b and consequently the transform between the two is the identity, leaving only $T_{b@t_1}^{c@t_1}$ needing to be computed to find the current location of any previously observed object. This simple case is common in many robotic systems because they do not have the tools to compute the more complicated cases. This will only work when the assumptions about where the identity transform can be used is maintained.